

Algoritmi e Strutture Dati – 16/06/2021

Esercizio A1 – Punti ≥ 8

1. $f(n) = n$: si può applicare il teorema base, con $\alpha = 2$, $\beta = 1$, siamo nel caso (1) quindi la complessità è $\Theta(n^2)$.
2. $f(n) = n \log n$: si deve applicare la versione estesa, $\alpha = 2$, $n \log n = O(n^{\alpha-\epsilon})$ per $\epsilon < 1$, siamo nel caso (1) quindi la complessità è $\Theta(n^2)$.
3. $f(n) = n^2$: si può applicare il teorema base, con $\alpha = 2$, $\beta = 2$, siamo nel caso (2) quindi la complessità è $\Theta(n^2 \log n)$.
4. $f(n) = n^2 + \log n$: si deve applicare la versione estesa, ma è facile vedere che $f(n) = \Theta(n^2)$, siamo nel caso (2) quindi la complessità è $\Theta(n^2 \log n)$.
5. $f(n) = n^2 / \log n$: si deve applicare la versione estesa, ma non rientra in nessuno dei tre casi, quindi la risposta è N/A.
6. $f(n) = n^2 \log n$: dobbiamo usare la versione estesa, ma non rientra in nessuno dei tre casi, quindi la risposta è N/A.
7. $f(n) = n^3$: si può applicare il teorema base, con $\alpha = 2$, $\beta = 3$, siamo nel caso (3) quindi la complessità è $\Theta(n^3)$.
8. $f(n) = n^3 \log n$: dobbiamo usare la versione estesa; è facile vedere che $n^3 \log n$ è $\Omega(n^{2+\epsilon})$. Bisogna però anche dimostrare che $\exists c : 0 < c < 1, \exists m \geq 0 : af(n/b) \leq cf(n), \forall n \geq m$. Quindi:

$$\begin{aligned} 16(n/4)^3 \log n / 4 &= \\ \frac{1}{4} n^3 (\log n - \log 4) &\leq \\ \frac{1}{4} n^3 \log n &\leq cn^3 \log n \end{aligned}$$

L'ultima disequazione è vera per $c = 1/4$. La complessità è quindi $\Theta(n^3 \log n)$.

Esercizio A2 – Punti ≥ 10

Questo esercizio è una variante dell'Esercizio 2 del 07/01/13. Invece di essere collocati in un unico vettore ordinato, i valori sono ordinati in due vettori distinti.

Una possibile soluzione consiste quindi nell'unire i due vettori in unico vettore ordinato grande $2n$, utilizzando una procedura simile alla `merge()` di `mergesort()` (costo $O(n)$) e poi chiamare la soluzione di quell'esercizio (con costo $O(n)$), per un costo totale $O(n)$.

Vediamo invece una soluzione ad-hoc per risolvere questa particolare versione del problema; per il principio di funzionamento, fate riferimento alla soluzione del 07/01/13.

```
int closePair(int[] A1, int[] A2, int n, int k)
```

```
int i1 = 1
int i2 = n
boolean found = false
while not found and i1 ≤ n and i2 ≥ 1 do
    sum = A1[i1] + A2[i2]
    if sum == k then
        found = true
    else if sum > k then
        i2 = i2 - 1
    else
        i1 = i1 + 1
return found
```

La complessità dell'algoritmo è $O(n)$, perché i due indici possono scorrere da 1 a n e da n a 1.

- Una soluzione alternativa consiste nell'inserire tutti gli elementi di A_1 in un insieme implementato con tabella hash; il costo totale di inserimenti è $O(n)$. Dopo di che, per ogni elemento $a \in A_2$, si cerca se $k - a$ è presente in tabella; costo totale $O(n)$, valutata 100%.
- Una soluzione più inefficiente consiste nell'utilizzare una ricerca dicotomica; per ogni elemento $a \in A_1$, si cerca $k - a$ in A_2 in tempo $O(\log n)$. Costo totale $O(n \log n)$. Questa soluzione è stata valutata 80%.
- Una soluzione ancora più inefficiente consiste nel testare tutte le coppie con due cicli annidati; una simile soluzione ha costo $O(n^2)$. Questa soluzione è stata valutata al 60%.

Esercizio A3 – Punti ≥ 12

Sfruttiamo la memorizzazione dell'albero binario tramite vettore heap per verificare che l'albero possa essere memorizzato in un vettore di dimensione n e che tutte le caselle siano piene. Innanzitutto, utilizziamo la procedura `count()` definita nelle slide per contare il numero di nodi n .

Il meccanismo per calcolare la posizione di un nodo è il seguente: la radice ha posizione 1; un nodo i ha i figli in posizione $2i$ e $2i + 1$. Se un nodo ha posizione $i > n$, vuole dire che l'albero non è completo, perchè non è memorizzabile in un heap di n nodi.

La funzione `isComplete()` chiama la funzione `isInside()`, che restituisce **true** se il nodo T può essere scritto in posizione i -esima, con $i \leq n$, e se i suoi sottoalberi possono essere contenuti correttamente nello heap; **false** altrimenti.

```
boolean isComplete(TREE T, int n)
```

```
int n = count(T)
return isInside(T, 1, n)
```

```
boolean isInside(TREE T, int i, int n)
```

```
if T == nil then
    return true
else if i > n then
    return false
else
    return isInside(T.left, 2i, n) and isInside(T.right, 2i + 1, n)
```

La complessità è quella di due visite in profondità, quindi $\Theta(n)$.